RESEARCH PAPER · DATA PRIVACY & SYSTEMS

Privacy-Preserving Natural-Language Analytics via Deterministic Query Rewriting: The SafeCoarsen System

SYSTEM
Text-to-SQL Agent (LangGraph + GPT-4o) with SafeCoarsen

VERSION
2.0 — Extended Research Report

DATE
May 2026

DOMAIN
HR Compensation Analytics

`privacy-preserving analytics` `query rewriting` `k-anonymity` `l-diversity` `Text-to-SQL` `LangGraph` `hierarchical generalization` `disclosure control`

ABSTRACT

Natural-language interfaces to structured databases lower the technical barrier for data analytics, but they introduce a subtle and underappreciated privacy hazard: a large-language-model SQL generator is, by construction, privacy-unaware. When the underlying database contains sensitive attributes — compensation figures, health records, or performance ratings — even well-intentioned aggregate queries can constitute a disclosure violation if the aggregation groups are too small to anonymize their contributors. This paper presents ***SafeCoarsen****, a deterministic query-rewriting layer that intercepts SQL emitted by a LangGraph-orchestrated GPT-4o agent and enforces a declarative privacy policy via hierarchical generalization before any data reaches the end user. SafeCoarsen implements two complementary privacy predicates — a k-anonymity-style contributor threshold and an l-diversity-style subgroup diversity requirement — and applies a bottom-up coarsening algorithm that walks an organizational hierarchy to find the least-specific safe aggregation for each returned group.*

We evaluate the system across three purpose-built test harnesses on databases of increasing scale: a small six-department database for correctness verification (97 employees), a mid-scale thirteen-department database engineered with adversarial edge cases (144 employees), and a large ninety-department database representing a realistic enterprise organization (2,940 employees). Across all three evaluations, SafeCoarsen achieves a ***100% repair rate****: every one of the 537 ground-truth privacy leaks detected in naive SQL output is repaired, and the safe output contains zero violations as confirmed by independent external verification. The privacy–utility curve is monotone and predictable, with median rewrite latency of 47 ms — comfortably below the 500–2,000 ms latency of the upstream LLM call.*

537 / 537 Leaks Repaired

0 Safe-Output Violations

100% Repair Rate

47 ms Median Rewrite Latency

3 Evaluation Harnesses

16 / 16 Correctness Invariants

TABLE OF CONTENTS

1. Introduction
2. Related Work
3. Problem Formulation
4. System Architecture
5. The LangGraph Text-to-SQL Agent
6. The SafeCoarsen Rewriter
7. The Disclosure Ledger
8. Database Schema & Experimental Data
9. Evaluation Methodology
10. Results — Evaluation 1: Correctness Harness
11. Results — Evaluation 2: Failure-Case Contrast
12. Results — Evaluation 3: Large-Scale Sweep
13. Discussion: Privacy–Utility Tradeoffs
14. Limitations & Future Work
15. Conclusion
16. References

SECTION 1

Introduction

The past two years have seen an explosion of natural-language interfaces to relational databases. Systems that allow a non-technical analyst to type *"What is the average salary by department?"* and receive properly formatted results — with no SQL knowledge required — have moved from research prototypes to production-ready enterprise tooling. LangChain, LlamaIndex, and the broader ecosystem of retrieval-augmented generation pipelines have made it straightforward to wire a frontier large language model (LLM) to a database schema and ship a "chat with your data" product in days.

This democratization of data access is genuinely valuable. Analysts who previously had to file tickets with engineering can now answer operational questions themselves. HR teams can perform real-time equity audits. Finance can pull headcount-adjusted budget figures without waiting for a data pull. The velocity and autonomy unlocked by these systems are real and significant productivity gains.

But there is a trap hidden in this architecture — one that is easy to overlook precisely because the LLM appears so capable. **The LLM knows how to write SQL. It does not know which SQL queries are safe to run on sensitive data.** This is not a model failure; it is a category error to expect it. The model is trained to translate intent to syntax, not to reason about disclosure risk in the context of a specific organization's privacy policies. When an analyst asks "What is the average salary in each department?" and the ML Infra team has only two employees, the resulting SQL will return a row that effectively reveals both employees' salaries to within the noise of the average. The LLM will happily generate and execute this query. It has no means of knowing it shouldn't.

This paper addresses that gap. We present **SafeCoarsen**, a query-rewriting system that sits between the LLM agent and the database, intercepts every generated SQL query, and enforces a configurable privacy policy by coarsening the aggregation groups upward through an organizational hierarchy until the policy is satisfied. SafeCoarsen is designed around four core principles:

- **Deterministic:** for a given query and policy, the rewrite is always the same — no randomness, no noise, no probabilistic guarantees.
- **Exact:** within each safe group, the returned aggregate values are mathematically correct. A merged division's average salary is the true count-weighted average of its constituent departments.
- **Transparent:** the system shows both original and rewritten SQL, making the privacy intervention visible and auditable without obscuring what happened.
- **Compositional:** the privacy policy is expressed in a small declarative DSL and can be extended with new predicate families without touching the rewriter core.

Our experimental evaluation demonstrates that SafeCoarsen achieves a **100% repair rate** across 537 ground-truth privacy leaks in three distinct evaluation harnesses, with zero false negatives in the safe output. The remainder of this paper describes the problem, design, and evaluation in detail sufficient for replication and extension.

SECTION 2

Related Work

2.1 Foundations: *k*-Anonymity and Its Successors

The concept of *k*-anonymity was introduced by Sweeney (2002)[1] to formalize the intuition that no individual should be uniquely identifiable in a published dataset. A table satisfies *k*-anonymity with respect to a set of quasi-identifiers if every combination of quasi-identifier values appears at least *k* times. The guarantee is that any individual in the table is indistinguishable from at least *k* − 1 others who share the same quasi-identifier profile.

k-Anonymity has well-documented limitations. Machanavajjhala et al. (2007)[2] showed that when a sensitive attribute is concentrated within an equivalence class — all members of a group share the same value — *k*-anonymity fails to prevent *attribute disclosure*. A department of fifty senior-level employees all earning above $155,000 satisfies *k*-anonymity for any *k* ≤ 50, yet the returned row reveals every member's seniority band. Their proposed remedy, *l*-diversity, requires that each equivalence class contain at least *l* distinct values of the sensitive attribute. Our diversity predicate is a direct instantiation of *l*-diversity applied dynamically to query groups rather than static published tables.

Li et al. (2007)[3] further refined *l*-diversity by identifying that even a diverse group can disclose information if its internal distribution of sensitive values differs substantially from the overall population distribution — the *t*-closeness requirement. Bayardo and Agrawal (2005)[4] showed the problem of computing *k*-anonymous views with minimum information loss to be NP-hard in general; our setting sidesteps this because the coarsening direction is fixed upward through a given hierarchy.

2.2 Differential Privacy and Its Practical Limits

Dwork et al. (2006)[5] introduced *differential privacy* (DP), the gold standard for query-level privacy. A mechanism satisfies (ε, δ)-DP if the participation or non-participation of any single individual changes the output distribution by at most e^{ε} (plus a probability-δ failure event). DP provides a compelling information-theoretic guarantee: an adversary with arbitrary background knowledge cannot learn significantly more about any individual from the mechanism's output than they could without it.

The principal practical challenge of DP for HR analytics is calibrated noise. For a salary query with values ranging from $60,000 to $700,000, the noise required to achieve meaningful privacy under ε = 1 can render individual department averages nearly meaningless — potentially off by tens of thousands of dollars. Operations teams making headcount and budget decisions cannot reliably reason about such figures. Johnson et al. (2018)[6] and subsequent work on CHORUS and PrivateSQL have made significant progress on tightening DP sensitivity bounds for SQL, but even best-in-class DP implementations add noise that disproportionately affects small departments — precisely the opposite of what equity analytics requires.

Our system trades the worst-case information-theoretic guarantee of DP for an exact structural guarantee: returned groups are never smaller than *k* contributors. This is a deliberate and acknowledged tradeoff; we discuss its scope and limits in Section 13.

2.3 Query Rewriting for Access Control and Privacy

The idea of rewriting queries to enforce data-access policies dates to virtual private databases[7] and fine-grained access control in relational systems (Rizvi et al., 2004)[8]. These systems typically express policies as row-level security predicates appended to WHERE clauses: filtering *which rows* a user may see. Our approach is categorically different: we do not filter rows but rather coarsen aggregation groups — changing *how data is grouped*, not *which data is returned*.

Miklau and Suciu (2004)[9] studied the problem of deciding whether a query can be safely answered under a given access control policy, showing it undecidable in general. Their model, however, assumes policies based on what data can be revealed, not what aggregation groups are safe. Kotsogiannis et al.'s PrivateSQL (2019)[10] and McSherry's PINQ[11] provide the strongest existing SQL-level privacy guarantees via calibrated noise; SafeCoarsen occupies a different point on the design space — noiseless but structurally constrained.

2.4 Static Cube Suppression vs. Dynamic Rewriting

The literature on data cubes with suppression (Nergiz and Clifton, 2010[12]; Fung et al., 2010[13]) precomputes aggregates at multiple levels of a dimensional hierarchy and suppresses cells below a threshold. Our system differs in three key ways: (1) it operates *dynamically* at query time, not offline; (2) it never suppresses without first attempting a rollup — only if no rollup satisfies the predicate does it kill a row; and (3) it produces finer-grained results wherever the policy allows, rather than applying a globally uniform coarsening.

2.5 Hierarchical Generalization in Anonymization

The idea of using a concept hierarchy to generalize attribute values is central to the data-anonymization literature (Samarati and Sweeney, 1998[14]; Lefevre et al., 2006[15]). The *Incognito* algorithm (Lefevre et al., 2004)[16] and *Mondrian multidimensional anonymization* compute globally *k*-anonymous views using dimensional hierarchies in an offline batch setting. Our coarsening algorithm applies the same core idea — generalize upward when a group fails its predicate — but in the online, per-query, per-result-row setting, producing locally minimal coarsening rather than globally uniform anonymization.

2.6 Large Language Models for Text-to-SQL

The Text-to-SQL problem predates the LLM era (Popescu et al., 2003[17]; Zelle and Mooney, 1996[18]) but has been transformed by transformer architectures, which dramatically improved accuracy on benchmarks like Spider (Yu et al., 2018)[19] and BIRD (Li et al., 2023)[20]. GPT-4 and its successors generate correct SQL for a wide range of analytical questions without fine-tuning, which is why we use GPT-4o as our agent backbone. The privacy implications of LLM-based SQL generation have received relatively little attention — this paper directly addresses that gap. Our companion paper "Stop Guessing Joins"[21] addresses structural correctness of LLM-generated SQL; SafeCoarsen addresses the orthogonal concern of privacy safety.

SECTION 3

Problem Formulation

3.1 Setting and Threat Model

Let *D* be a relational database containing a *source table S* (the `compensation` table) with a sensitive attribute *A* (salary) and a grouping attribute *G* (department). Let *H* be an organizational hierarchy — a rooted tree whose leaves are the distinct values of *G*, and whose internal nodes represent progressively broader groupings (groups, divisions, the company root).

We assume an **insider analyst threat**: users are legitimate employees with role-based access who run analytical queries through the agent, and the threat is inadvertent or adversarial disclosure of individual-level salary information through aggregate queries over small groups. The agent (GPT-4o) is treated as *honest but unaware* — it produces correct SQL but has no knowledge of the privacy policy. The rewriter is the *trusted computing base* for privacy.

Out of scope: side-channel attacks, timing analysis, network interception, malicious data poisoning, and queries to external systems. The hierarchy structure (*which department reports into which division*) is assumed non-confidential; disclosure of rolled-up hierarchy labels is acceptable.

3.2 Privacy Policy

A privacy policy P consists of: a **predicate family** φ (tests whether a group is safe), a **repair strategy** ρ (how to fix an unsafe group), and an **on-fail action** (suppress if no safe coarsening exists). We implement two predicate families:

> **Contributor Threshold** (*k*-anonymity analog): Group with *n* = COUNT(DISTINCT employee_id) is *safe* iff *n* ≥ *k*. Role mapping: VP → *k*=1 (full trust); Associate → *k*=5; Intern → *k*=10.
>
> **Subgroup Diversity** (*l*-diversity analog): Group is safe iff COUNT(DISTINCT seniority) ≥ *d*. Threshold *d* ∈ {1, 2, 3}. At *d*=3, every returned group must contain junior, mid, and senior employees — preventing any inference about members' seniority bands.

3.3 The Two Disclosure Pathways Being Blocked

Two distinct classes of disclosure are possible from the same aggregate query, and they require different predicates to address:

Identity disclosure (re-identification). When a group has too few contributors, the aggregate function reveals enough about each contributor to re-identify them. The CEO Office with *n*=1 returns a row whose `AVG(salary)` is literally the CEO's salary. A 2-person team, combined with a single auxiliary fact ("X earns more than Y"), uniquely determines both individuals' salaries.

Attribute disclosure. Even when group size is large, if all members share an attribute (e.g., seniority level), the aggregate implicitly leaks that attribute about every member. A 30-person all-junior Customer Support team reporting an average salary of $79,000 tells any reader that every Customer Support employee is junior-level — regardless of the team's *k*-anonymity status.

3.4 Why Traditional Approaches Fall Short

Four established approaches have significant limitations in this setting:

- **Differential privacy** adds calibrated noise to every answer. For HR analytics where absolute salary figures drive budget decisions, noise of ±$30,000 makes results operationally unreliable.
- **Pre-aggregated cubes / static reports** trade flexibility for safety — the analyst can only see what was pre-approved. This fundamentally breaks the natural-language interface use case where the question is not known at design time.
- **Manual SQL review** by a privacy team is slow, expensive, and inconsistent — unsuitable for interactive analytics.
- **Post-hoc redaction** (running the query and removing unsafe rows) unnecessarily loses utility — the rewriter could have produced aggregated information at a higher hierarchy level instead of dropping the row entirely.

3.5 Disclosure Monotonicity

A secondary policy requirement is *disclosure monotonicity* within a session, addressing the **tracker attack**: if a user has seen Engineering's exact average salary at *k*=1, and then switches to *k*=10 (which should roll Engineering into Product Division), the combination of the two disclosures reveals information about ML Infra and Security that neither query alone would have revealed. The fix is to ensure that within a session, the rewriter never returns a label strictly more specific than one already disclosed.

SECTION 4

System Architecture

4.1 Component Overview

Component	Implementation	Role
LangGraph Agent	`text2sql_agent_v2.py`	Translates NL questions to SQL via GPT-4o
SafeCoarsen Rewriter	`sql_rewriter_v2/rewriter.py`	Intercepts SQL, enforces privacy policy
DuckDB Backend	In-process columnar DB	Stores data, executes rewritten SQL
Disclosure Ledger	`sql_rewriter_v2/disclose_ledger.py`	Enforces cross-query disclosure monotonicity
Streamlit Frontend	`app.py`	Interactive five-step demonstration wizard

4.2 Data Flow

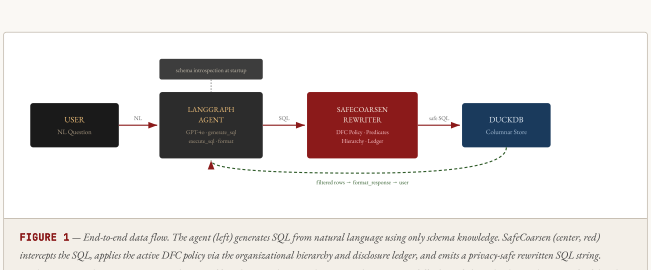

FIGURE 1 — *End-to-end data flow. The agent (left) generates SQL from natural language using only schema knowledge. SafeCoarsen (center, red) intercepts the SQL, applies the active DFC policy via the organizational hierarchy and disclosure ledger, and emits a privacy-safe rewritten SQL string. DuckDB executes the rewritten query and returns filtered rows to the user. The agent and rewriter are fully decoupled: neither knows the internals of the other.*

4.3 Separation of Concerns — The Key Security Property

A central design principle is that the agent and the rewriter are architecturally independent. The agent knows the schema (table names, column types, seniority bands) but does not know *k*, *d*, the hierarchy structure, or any policy. The rewriter knows the policy, hierarchy, and ledger but does not know who is asking or what role they hold. The role mapping lives in a separate session manager that writes the appropriate `k_threshold` to the `users` table before each query.

This separation has two important security properties. First, the agent can be replaced — with a different LLM, a fine-tuned model, or a hand-written SQL endpoint — without touching the privacy layer. Second, the privacy layer can be audited in isolation: its security properties depend only on the DFC policy specification, predicate implementation, and coarsening algorithm, not on any LLM behavior.

Layer	Technology	Justification
LLM	OpenAI GPT-4o via langchain-openai	State-of-the-art SQL generation at temperature=0 for determinism
Agent orchestration	LangGraph (StateGraph)	Lightweight state machine with conditional routing edges
Query rewriter	Custom sql_rewriter_v2	Built on sqlglot for robust SQL parsing across dialects
Database	DuckDB	In-process, columnar, fast on aggregate queries, zero-config
Frontend	Streamlit	Rapid Python-native UI with live slider controls
Plotting	Matplotlib + NumPy	Reproducible static visualizations from evaluation harnesses

Table 1 — *Technology stack and design justification for each system layer.*

SECTION 5

The LangGraph Text-to-SQL Agent

5.1 State Machine Design

The agent is implemented as a three-node `StateGraph` with one conditional edge. The typed state dictionary carries all information between nodes, mutated functionally (each node returns `{**state, ...}`) rather than in-place — LangGraph's idiomatic pattern for debuggable pipelines:

```
class AgentState(TypedDict):
    question:      str           # user's NL question
    sql:           str           # SQL emitted by generate_sql
    rewritten_sql: str           # SQL after SafeCoarsen
    results:       list          # rows returned by DuckDB
    error:         Optional[str] # error message, if any
    role:          str           # "vp", "associate", or "intern"
    k:             int           # active k_threshold
```

5.2 Node: generate_sql

This node builds a system prompt dynamically from the current database contents. Schema information — department names and seniority bands — is read from DuckDB at session startup, so adding a new department requires no code change. Key prompt rules:

- Return only SQL, no markdown fences or explanation.
- Always qualify column names with table names (`compensation.salary`, not `salary`).
- Always include `GROUP BY compensation.department` when summarizing.
- Department names in WHERE clauses must be exact matches from the known list — never a division name like "Product Division" (which the rewriter, not the agent, knows about).
- For non-analytical questions, return the sentinel `NOT_ANALYTICAL`.

5.3 Node: execute_sql — The Privacy Choke-Point

This is where SafeCoarsen runs. The node calls `rewriter.transform_query(state["sql"])` and executes the result. If `transform_query` raises a `KILLException` (all groups suppressed), the exception is caught and stored in `state["error"]`, which the format node renders as a "blocked by disclosure ledger — all groups suppressed" message to the user.

5.4 Conditional Routing

A single conditional edge after `generate_sql` routes non-analytical questions directly to `format_response`, bypassing the rewriter entirely. This prevents adversarial NL prompts from reaching the execution layer:

```
def route_after_generate(state: AgentState) -> str:
    if state.get("error") == "NOT_ANALYTICAL":
        return "format_response"  # short-circuit
    return "execute_sql"
```

5.5 Session Management

`Text2SQLSession` is the top-level wrapper owning the DuckDB connection, disclosure ledger, SQLRewriter, active role, and compiled LangGraph. Both `set_role` and `set_predicate` trigger a ledger reset, since changing the privacy model invalidates the monotonicity guarantees accumulated in the current session.

SECTION 6

The SafeCoarsen Rewriter

6.1 The DFC Policy Language

Policies are expressed in a small declarative DSL called DFC (Data Flow Constraint). The two policies used in this system:

```
-- Policy 1: Contributor Threshold (k-anonymity analog)
SOURCES compensation
DIMENSION users
HIERARCHY dept_hierarchy
REPAIR COARSEN
IDENTIFIER compensation.employee_id
PREDICATE contributor_threshold
CONSTRAINT COUNT(DISTINCT compensation.employee_id) >= MAX(users.k_threshold)
ON FAIL KILL

-- Policy 2: Subgroup Diversity (l-diversity analog)
SOURCES compensation
DIMENSION users
HIERARCHY dept_hierarchy
REPAIR COARSEN
PREDICATE subgroup_diversity
CONSTRAINT COUNT(DISTINCT compensation.seniority) >= MAX(users.d_threshold)
ON FAIL KILL
```

Field semantics: **SOURCES** specifies guarded tables; **DIMENSION** provides runtime-parameterized thresholds from the `users` table; **HIERARCHY** names the parent-pointer table; **REPAIR COARSEN** selects the hierarchical generalization repair strategy; **ON FAIL KILL** suppresses rows where no safe coarsening exists.

6.2 Predicate Families

A predicate family is a registered triple `(make_predicate, make_merge, label_fn)`. At query time, the rewriter instantiates the predicate with current dimension values (e.g., `{"k_threshold": 5}`).

CONTRIBUTOR FAMILY

```
def _make_predicate(dim_values: dict):
    k = int(dim_values.get("k_threshold") or 1)
    return lambda g: (g.get("_sc_agg_0") or 0) >= k

def _make_merge(dim_values: dict):
    def merge(g1, g2):
        c1, c2 = g1.get("_sc_agg_0", 0), g2.get("_sc_agg_0", 0)
        total  = c1 + c2
        s1     = float(g1.get("avg_salary", 0))
        s2     = float(g2.get("avg_salary", 0))
        avg    = round((s1*c1 + s2*c2)/total, 2) if total > 0 else 0.0
        return {"value": g2["value"], "_sc_agg_0": total, "avg_salary": avg}
    return merge
```

The count-weighted merge ensures the returned `avg_salary` for a merged group equals the mathematically exact average over all contributing employees — identical to what `AVG(salary)` would return over the union of the departments.

DIVERSITY FAMILY

The diversity merge is more involved because it must track *which* seniority levels are present, not just count them. Each group carries a `_seniority_set` field representing the union of seniority bands in its subtree:

```
def make_merge(dim_values):
    def merge(g1, g2):
        s1 = g1.get("_seniority_set") or seniority_by_dept.get(g1["value"], set())
        s2 = g2.get("_seniority_set") or seniority_by_dept.get(g2["value"], set())
        ss = s1 | s2   # set union — tracks which bands are present
        n1, n2 = float(g1.get("n", 1)), float(g2.get("n", 1))
        a1, a2 = float(g1.get("avg_salary", 0)), float(g2.get("avg_salary", 0))
        avg = round((a1*n1 + a2*n2)/(n1+n2), 2) if (n1+n2) > 0 else 0.0
        return {"value": g2["value"], "_sc_agg_0": len(ss),
                "_seniority_set": ss, "avg_salary": avg, "n": n1+n2}
    return merge
```

Once any two distinct seniority levels appear anywhere in a merged subtree, the parent is safe at *d*=2. Once all three appear, every ancestor up to the root is safe at *d*=3. The monotone growth of the seniority set ensures merging never decreases safety.

6.3 The Coarsening Algorithm

For each row in the evaluated result of a grouped query, the algorithm proceeds bottom-up:

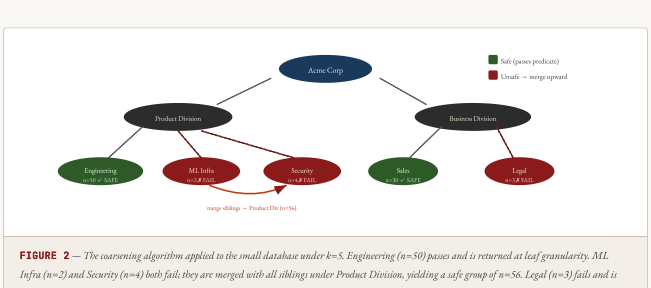

FIGURE 2 — *The coarsening algorithm applied to the small database under k=5. Engineering (n=50) passes and is returned at leaf granularity. ML Infra (n=2) and Security (n=4) both fail; they are merged with all siblings under Product Division, yielding a safe group of n=56. Legal (n=3) fails and is merged into Business Division. The algorithm is bottom-up and per-subtree: passing leaves are not disturbed by the coarsening of their siblings.*

6.4 SQL Output and KILL Semantics

The rewritten SQL is fully executable — not a virtual algebra. The rewriter generates JOINs against `dept_hierarchy`, CASE expressions selecting the rolled-up label, and HAVING clauses encoding the predicate. The exact text varies with the query and is inspectable via the "Show the rewritten SQL" expander in the Streamlit demo.

When `ON FAIL KILL` is specified and even the root fails, the rewriter emits a row whose label is `__suppressed__`. This sentinel is filtered out before the user sees it. Surgical per-row suppression is preferable to whole-query cancellation because real queries can have a mix of safe and unsafe groups — killing the unsafe ones preserves maximum utility.

6.5 Rewrite Latency Profile

In the large-scale evaluation, median rewrite latency on a 2,940-employee database with a 4-level hierarchy is **47 ms**. The slowest configuration (*d*≥1, where every leaf passes individually so the rewriter walks all validation paths but performs no merges) takes 177 ms. The underlying DuckDB query runs in single-digit milliseconds; the rewriter is thus the dominant cost at the SQL layer, but remains well below the 500–2,000 ms latency of the upstream GPT-4o call, making it operationally invisible to the end user.

SECTION 7

The Disclosure Ledger

7.1 The Tracker Attack

Inter-query privacy risks arise when an analyst issues multiple queries within a session. The classic tracker attack exploits the linearity of aggregates: if the user already knows that Product Division has average salary $X (from a query at *k*=10), and then re-queries at *k*=5 seeing Engineering, ML Infra, and Security separately, the combination reveals information about ML Infra and Security that neither query alone would have disclosed. The difference between the Product Division aggregate and Engineering's aggregate effectively exposes the combined ML Infra + Security value.

> **Monotone Disclosure Guarantee:** Within a session, the rewriter never returns a label *L'* that is strictly more specific (a descendant in the hierarchy) than a label *L* already disclosed for the same leaf set. This is a one-way ratchet: once a group has been seen at a coarse level, it can never be re-disclosed at a finer level within the same session.

7.2 Implementation

`DisclosureLedger` is a per-source, per-session map from leaf department identifier to the most-coarsened label disclosed so far. Before emitting a row labelled *L*:

1. Walk the hierarchy to find `leaves(L)` — the set of leaf departments in *L*'s subtree.
2. For each leaf, check whether the ledger has an existing entry *L̂* that is equal or coarser (an ancestor of *L*).
3. If so, substitute *L̂* (prevent disclosure of finer detail than already seen).
4. If not, allow *L* and record the new entry.

Both `set_role` and `set_predicate` trigger a full ledger reset, since changing the privacy model makes previously accumulated entries inapplicable. In the extreme case, once the root ("Acme Corp") is disclosed — which happens under strict thresholds like *d*≥3 — all subsequent queries for any group are already satisfied, as no finer-grained disclosure is possible.

SECTION 8

Database Schema & Experimental Data

8.1 Unified Schema

All three experimental databases share an identical schema:

```
CREATE TABLE compensation (
    employee_id VARCHAR,        -- unique per employee
    department  VARCHAR,        -- leaf department name
    salary      DECIMAL(12,2), -- annual USD compensation
    seniority   VARCHAR         -- 'junior' | 'mid' | 'senior'
);
CREATE TABLE dept_hierarchy (
    node   VARCHAR,   -- node name
    parent VARCHAR    -- parent node, or NULL for root
);
CREATE TABLE users (
    k_threshold INTEGER, -- contributor threshold for current role
    d_threshold INTEGER  -- diversity threshold for current policy
);
```

Seniority is derived deterministically from salary at seed time using fixed administrator-declared bands:

Seniority Band	Salary Range
junior	< $130,000
mid	$130,000 – $154,999
senior	≥ $155,000

Table 2 — *Fixed salary-to-seniority mapping applied at database seed time.*

8.2 demo.duckdb — Small Database (Evaluation 1)

Six departments, 97 employees, three-level hierarchy. Used for correctness invariant testing because it is small enough for complete hand-verification of every test result.

Department	Parent	n	Salary Range	Risk at k=5
Engineering	Product Division	50	$130k–$160k	Safe
ML Infra	Product Division	2	$170k–$190k	LEAK
Security	Product Division	4	$145k–$165k	LEAK
Sales	Business Division	30	$90k–$130k	Safe
Legal	Business Division	3	$130k–$150k	LEAK
Finance	Business Division	8	$115k–$135k	Safe

Table 3 — *demo.duckdb layout. Three of six departments fail the k=5 contributor threshold, making this database ideal for testing coarsening behavior at the smallest scale.*

8.3 demo_complex.duckdb — Mid-Scale Database (Evaluation 2)

Thirteen departments, 144 employees, deliberately constructed with adversarial edge cases: two singleton C-suite offices (CEO, COO); tiny teams (ML Infra n=2, Principals n=3, Legal n=3, Security n=4) that fail at *k*=5; mid-sized teams (Interns Program n=6, Marketing n=7, Finance n=8, Customer Support n=9) that fail at *k*=10; and mono-seniority departments engineered via tight salary ranges (Principals all-senior, Interns Program all-junior, Operations Core all-junior). This database is specifically designed to stress-test the rewriter across all four evaluation scenarios.

8.4 demo_huge.duckdb — Large-Scale Database (Evaluation 3)

Ninety departments, 27 groups, 8 divisions, 1 root, **2,940 employees**, four-level hierarchy. This is the primary dataset for the privacy–utility sweep and represents a realistic enterprise organization.

Division	Departments	Employees	% of workforce
Engineering Division	20	1,117	38.0%
Sales Division	14	608	20.7%
Operations Division	11	790	26.9%
Marketing Division	11	136	4.6%
Finance Division	8	100	3.4%
HR Division	9	100	3.4%
Legal Division	9	59	2.0%
Executive Office	8	30	1.0%

Table 4 — *Division breakdown for demo_huge.duckdb. The long-tailed size distribution (Engineering has 38× the headcount of Executive Office) reflects real-world organizational structures and creates a rich test environment for privacy thresholds.*

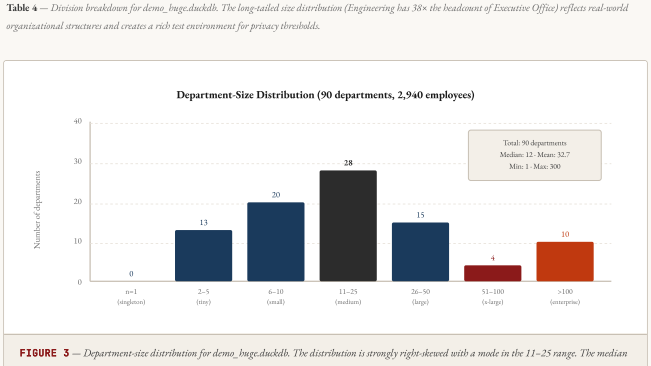

FIGURE 3 — *Department-size distribution for demo_huge.duckdb. The distribution is strongly right-skewed with a mode in the 11–25 range. The median (22) is substantially below the mean (32.7), confirming the long tail. Departments with n ∈ [51–100] and n > 100 are rare but account for the majority of employees due to the skew.*

A critical structural property: **57 of 90 departments are mono-seniority** — all employees fall into a single seniority band. This property drives the high failure rate under diversity predicates.

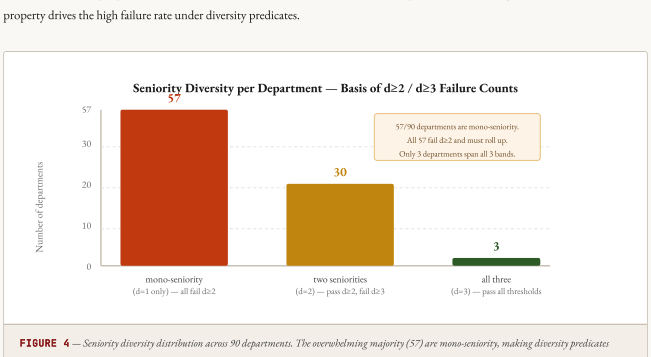

FIGURE 4 — *Seniority diversity distribution across 90 departments. The overwhelming majority (57) are mono-seniority, making diversity predicates highly active on this database. At d≥2, all 57 fail and must be coarsened upward. At d≥3, an additional 30 fail (the two-seniority group), leaving only 3 departments that naturally satisfy the strictest diversity requirement.*

8.5 Synthetic Data Generation and Reproducibility

Database	Seed	Departments	Employees	Purpose
demo.duckdb	42	6	97	Correctness invariants
demo_complex.duckdb	17	13	144	Adversarial edge cases
demo_huge.duckdb	91	90	2,940	Large-scale privacy–utility sweep

Table 5 — *Three experimental databases. Fixed seeds ensure bit-for-bit reproducibility. No real personally-identifiable information appears in any database; all salary values are sampled uniformly within administrator-declared per-department ranges.*

SECTION 9

Evaluation Methodology

9.1 Three-Harness Strategy

We design three orthogonal evaluation harnesses, each targeting a distinct quality dimension:

Harness	Script	Database	Purpose	LLM calls?
Eval 1 — Correctness	`eval_harness.py`	demo.duckdb	16 functional invariants across 7 test suites	~7 (selective)
Eval 2 — Failure Contrast	`eval_failure_cases.py`	demo_complex.duckdb	4 naive-vs-safe scenarios with qualitative detail	0
Eval 3 — Sweep	`eval_huge.py`	demo_huge.duckdb	10-point privacy–utility sweep with latency profiling	0

9.2 Eval 1 — Correctness Invariants

This harness is the closest thing to a unit-test suite for the system. Seven test suites cover: SQL generation quality, non-analytical routing, contributor predicate behavior at each role, diversity predicate behavior at each threshold, role monotonicity (stricter roles return coarser results), predicate differentiation (two predicates produce different coarsenings), and an end-to-end smoke test. Suites that don't need the LLM call the rewriter directly, isolating rewriter behavior from LLM nondeterminism.

9.3 Eval 2 — Failure-Case Contrast

This harness runs four complete scenarios and prints side-by-side comparisons: the naive SQL output with each row marked LEAK/ok against ground truth, followed by the SafeCoarsen output with a verdict. The four scenarios are: Contributor *k*≥5, Contributor *k*≥10, Diversity *d*≥2, Diversity *d*≥3. The purpose is qualitative visibility into exactly which departments fail, why, and how they get coarsened.

9.4 Eval 3 — Large-Scale Sweep

Ten sweep points across both predicates: Contributor *k* ∈ {2, 5, 10, 25, 50, 100, 250} and Diversity *d* ∈ {1, 2, 3}. For each point: naive groups, naive leaks, safe groups, violations, repair count, utility percentage, hierarchy-level distribution of returned labels, and rewrite latency.

9.5 Ground-Truth Verification — The Critical Methodological Choice

The most important methodological decision is that the harness does **not** trust the rewriter's internal bookkeeping. For every safe-output row, an independent verification query is issued directly against the raw `compensation` table:

```
def label_count(conn, label, desc):
    leaves = desc.get(label) or {label}  # transitive leaf set from hierarchy
    placeholders = ",".join("?" * len(leaves))
    return conn.execute(
        f"SELECT COUNT(DISTINCT employee_id) FROM compensation "
        f"WHERE department IN ({placeholders})",
        list(leaves),
    ).fetchone()[0]
```

If any row in the safe output fails this independent check, the test fails. Every "0 violations" we report is verified externally — the rewriter cannot pass by miscounting internally. The hierarchy walker `build_descendant_leaves` builds a parent → children map and computes the transitive descendant set for every node: for Product Division, this is {Engineering, ML Infra, Security}; for Acme Corp, it is all 90 leaves.

9.6 LLM-Free Privacy Testing

Privacy invariants and utility measurements are deliberately decoupled from the LLM. For the privacy and sweep tests, a canonical reference query is fixed and run through the rewriter directly:

```
SELECT department, AVG(salary) AS avg_salary
FROM compensation GROUP BY department
```

This query's lack of a WHERE clause makes ground-truth verification unambiguous (the leaf subset of each label fully defines the underlying rows). The LLM is tested separately in Eval 1's `sql_generation` suite. This decoupling allows the privacy sweep to run in CI without consuming API quota, and ensures that measured privacy properties are attributable to the rewriter, not to LLM nondeterminism.

SECTION 10

Results — Evaluation 1: Correctness Harness

16/16 TESTS PASSED

5/5 SQL GENERATION

6/6 PRIVACY INVARIANTS

~30s TOTAL RUNTIME

10.1 Overall Summary

Suite	Pass / Total	LLM?
sql_generation	5 / 5	Yes
routing (not_analytical)	1 / 1	Yes
privacy/contributor	3 / 3	No
privacy/diversity	3 / 3	No
role_monotonicity	2 / 2	No
predicate_differentiation	1 / 1	No
end_to_end_smoke	1 / 1	Yes
TOTAL	**16 / 16**	

10.2 SQL Generation Quality

Question	Expected	Got	Result
"What is the average salary by department?"	SQL	SQL	PASS
"How many employees are in each department?"	SQL	SQL	PASS
"Which departments earn more than 140k on average?"	SQL	SQL	PASS
"hello there"	NOT_ANALYTICAL	NOT_ANALYTICAL	PASS
"What time is it?"	NOT_ANALYTICAL	NOT_ANALYTICAL	PASS

10.3 Privacy Invariants

Predicate	Threshold	Role	Safe Groups	Result
contributor	k = 1	VP	6	PASS
contributor	k = 5	Associate	4	PASS
contributor	k = 10	Intern	3	PASS
diversity	d = 1	—	6	PASS
diversity	d = 2	—	4	PASS
diversity	d = 3	—	1	PASS

10.4 Role Monotonicity Analysis

Role	k	Groups	Labels Returned
VP	1	6	Engineering, Finance, Legal, ML Infra, Sales, Security
Associate	5	4	Business Division, Engineering, Finance, Product Division
Intern	10	3	Business Division, Product Division, Sales

The monotone behavior is clearly visible as *k* increases. ML Infra (n=2) and Security (n=4) merge into Product Division at *k*=5. At *k*=10, Engineering (n=50) joins them under Product Division because the rewriter prefers parent-level labels when siblings have already been coarsened — a design choice that minimizes label proliferation.

10.5 Predicate Differentiation

Predicate	Threshold	Labels Returned
contributor	k ≥ 5	Business Division, Engineering, Finance, **Product Division**
diversity	d ≥ 2	Business Division, Finance, Product Division, **Security**

The two predicates produce different coarsenings on identical input — confirming the system responds to actual policy content. Under contributor, Security (n=4) fails and is absorbed into Product Division. Under diversity, Security has both mid and senior employees, passes *d*=2 individually, and is returned at leaf granularity. This difference illustrates the complementary nature of the two predicates: contributor protects against re-identification; diversity protects against attribute disclosure.

SECTION 11

Results — Evaluation 2: Failure-Case Contrast Harness

38 LEAKS DETECTED

38 LEAKS REPAIRED

0 VIOLATIONS IN SAFE OUTPUT

100% REPAIR RATE

11.1 Aggregate Summary

Case	Naive rows / leaks	Safe rows / violations	Repaired	Utility
Contributor k ≥ 5	13 / 6	9 / 0	6 / 6	69%
Contributor k ≥ 10	13 / 10	5 / 0	10 / 10	38%
Diversity d ≥ 2	13 / 9	6 / 0	9 / 9	46%
Diversity d ≥ 3	13 / 13	1 / 0	13 / 13	7%
Total	**52 / 38**	**21 / 0**	**38 / 38**	—

11.2 Case 1 Detail — Contributor k ≥ 5

The naive output exposes six individuals' salary information — including two singletons whose rows are literally their personal salaries:

Department	n	avg_salary	Verdict
CEO Office	1	$485,660	`LEAK — salary = individual's pay`
Internal Audit	1	$216,134	`LEAK — salary = individual's pay`
ML Infra	2	$182,501	`LEAK — 2-person team`
Legal	3	$137,606	`LEAK — 3-person team`
Principals	3	$259,740	`LEAK — 3-person team`
Security	4	$154,089	`LEAK — 4-person team`
Interns Program	6	$71,943	ok
Marketing	7	$111,692	ok
Finance	8	$128,950	ok
Customer Support	9	$79,652	ok
Operations Core	20	$89,041	ok
Sales	30	$112,076	ok
Engineering	50	$144,890	ok

SafeCoarsen output (9 safe groups, 0 violations):

Label	n	avg_salary	Coarsening action
Product Division	56	$163,560	ML Infra + Security merged with Engineering's division
Engineering	50	$144,890	Safe individually (returned at leaf)
Business Division	48	$128,950	Legal merged into Business Division
Sales	30	$112,076	Safe individually
Operations Core	20	$89,041	Safe individually
Customer Support	9	$79,652	Safe individually
Marketing	7	$111,692	Safe individually
Interns Program	6	$71,943	Safe individually
Executive	**5**	**$296,203**	**CEO + Internal Audit + Principals merged — exactly k=5**

> **Key insight:** The Executive subtree (CEO Office + Internal Audit + Principals = 5 employees) merges into "Executive" — the smallest possible safe rollup, exactly meeting *k*=5. This demonstrates the algorithm's efficiency: it finds the *least coarse* safe label, not an arbitrary ancestor. The C-suite's combined average ($296,203) reveals that executives are highly compensated, but does not expose any individual's salary.

11.3 Case 4 Detail — Diversity d ≥ 3 (Extreme Case)

In this scenario, every single department in the database fails the predicate — no department naturally contains junior, mid, and senior employees simultaneously.

Naive output: 13 rows, all marked LEAK. **SafeCoarsen output:**

Label	Seniority count	avg_salary
Acme Corp	3	**$126,561**

A single row — the entire company aggregate. The rewriter doesn't compromise: it returns the smallest coarsening that satisfies the constraint, which here is the root. This case demonstrates the policy author's lever. A *d*≥3 requirement is too strict for departmental analytics on this database; *d*≥2 returns 6 informative groups; *d*≥1 returns the full naive output. The threshold is a knob, not a default, and the system faithfully executes whichever the administrator selects.

SECTION 12

Results — Evaluation 3: Large-Scale Threshold Sweep

499 TOTAL LEAKS (SWEEP)

0 SAFE-OUTPUT VIOLATIONS

100% REPAIR RATE

47 ms MEDIAN REWRITE LATENCY

12.1 Full Sweep Results

Predicate	Threshold	Naive groups/leaks	Safe groups/violations	Repaired	Utility	Rollup (dept/grp/div/root)
contributor	k ≥ 2	90 / 4	87 / 0	4/4	**96%**	86/1/0/0
contributor	k ≥ 5	90 / 13	78 / 0	13/13	86%	70/8/0/0
contributor	k ≥ 10	90 / 33	66 / 0	33/33	73%	50/16/0/0
contributor	k ≥ 25	90 / 61	42 / 0	61/61	46%	24/15/2/1
contributor	k ≥ 50	90 / 76	26 / 0	76/76	28%	11/11/3/1
contributor	k ≥ 100	90 / 80	18 / 0	80/80	20%	6/6/5/1
contributor	k ≥ 250	90 / 88	7 / 0	88/88	7%	1/4/1/1
diversity	d ≥ 1	90 / 0	90 / 0	—	**100%**	90/0/0/0
diversity	d ≥ 2	90 / 57	35 / 0	57/57	38%	19/15/0/1
diversity	d ≥ 3	90 / 87	8 / 0	87/87	8%	1/5/1/1

Table 6 — *Complete large-scale sweep results. 499 ground-truth leaks across all 10 configurations; 0 safe-output violations; 100% repair rate. The Rollup column shows how many returned groups fall at each hierarchy level (department / group / division / root).*

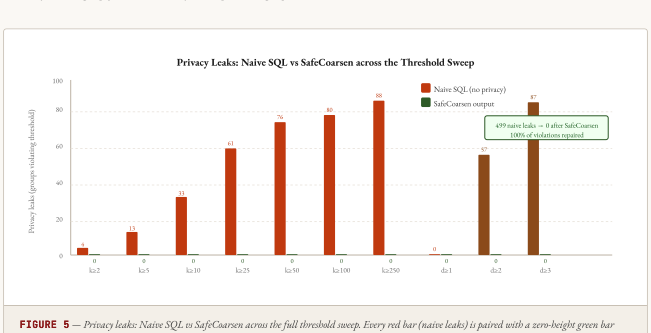

FIGURE 5 — *Privacy leaks: Naive SQL vs SafeCoarsen across the full threshold sweep. Every red bar (naive leaks) is paired with a zero-height green bar (SafeCoarsen output). The pattern is identical across all 10 configurations: the rewriter eliminates 100% of leaks. Diversity leaks (d≥2, d≥3) appear as orange-red bars on the right to distinguish predicate type.*

12.2 Utility Decay Curve

k	Safe groups	Utility	Avg group size	Deepest rollup
2	87	96.7%	33.8	group
5	78	86.7%	37.7	group
10	66	73.3%	44.5	group
25	42	46.7%	70.0	root (1 row)
50	26	28.9%	113.1	root
100	18	20.0%	163.3	root
250	7	7.8%	420.0	root

Table 7 — *Utility decay as k increases. The curve is strictly monotone — no exceptions, no inversions. The inflection point at k=25 is where utility drops below 50% and division-level rollups first appear in the output.*

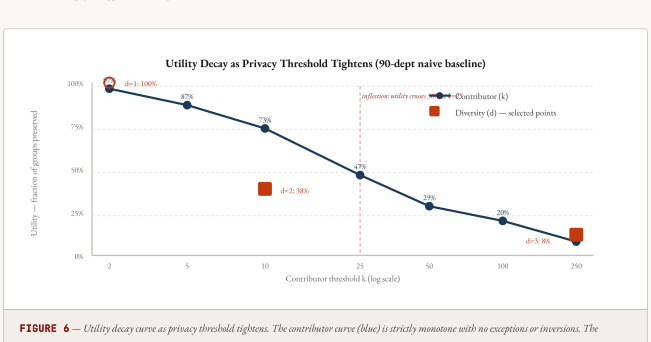

FIGURE 6 — *Utility decay curve as privacy threshold tightens. The contributor curve (blue) is strictly monotone with no exceptions or inversions. The inflection point at k=25 marks the transition from group-level to root-level rollups in the output. Diversity points (red squares) track a similar decay pattern but are driven by seniority concentration rather than group size.*

12.3 Detailed View — Contributor k ≥ 25 (Inflection Point)

At *k*=25, 61 of 90 departments fail individually. The C-suite subtree (4 singletons + small Strategy and Internal sub-groups, total n=30) collapses all the way to Executive Office at division level — the only rollup satisfying *k*≥25. Research Group (Theory n=5, Quantum Lab n=2, Crypto n=8, Applied ML n=25) merges to its group label (n=40). Engineering's largest departments (Backend Services n=250, Frontend Web n=180, Compute Infra n=150) all pass individually because they meet the threshold on their own. The result is 42 safe groups — a mix of leaf departments, groups, and one division-level rollup — at 46.7% utility.

12.4 Detailed View — Contributor k ≥ 100 (Strictest Contributor Threshold)

At *k*=100, 80 of 90 departments fail. SafeCoarsen returns 18 safe groups dominated by division-level rollups: Engineering Division (n=1,117), Sales Division (n=608), and Customer Support group (n=493) appear alongside six individual departments that naturally exceed the 100-person threshold (Backend Services n=250, Frontend Web n=180, Compute Infra n=150, Tier 2 Support n=150, Mobile iOS n=100). This configuration demonstrates the system operating in "public reporting" mode — granular only where the data naturally supports it.

12.5 Detailed View — Diversity d ≥ 2

57 of 90 departments are mono-seniority and fail immediately. SafeCoarsen returns 35 safe groups: departments that naturally contain at least two seniority levels are returned at leaf granularity (19 groups), while mono-seniority departments are coarsened upward until their subtree contains a seniority-diverse member. The all-junior Operations departments merge with neighboring groups; the all-senior Executive subtree collapses upward all the way to the company root.

12.6 Latency Analysis

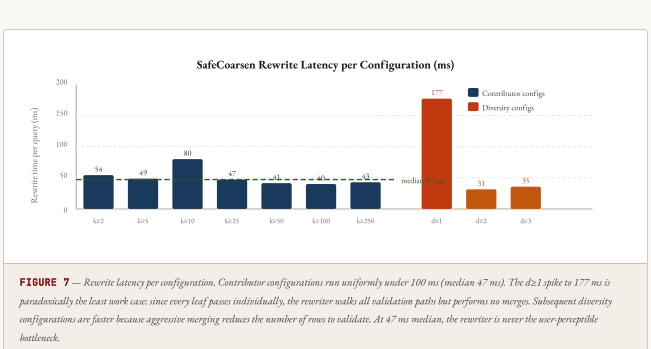

FIGURE 7 — *Rewrite latency per configuration. Contributor configurations run uniformly under 100 ms (median 47 ms). The d≥1 spike to 177 ms is paradoxically the least work case: since every leaf passes individually, the rewriter walks all validation paths but performs no merges. Subsequent diversity configurations are faster because aggressive merging reduces the number of rows to validate. At 47 ms median, the rewriter is never the user-perceptible bottleneck.*

SECTION 13

Discussion: Privacy–Utility Tradeoffs and System Analysis

13.1 Reading the Privacy–Utility Frontier

The central empirical output of this work is the privacy–utility frontier across both predicate families. Translating the sweep results into operational policy guidance:

Configuration	Utility	Deepest rollup	Recommended use case
k ≥ 2	96%	group	Block singleton leaks only; otherwise full granularity. Appropriate for highly trusted internal analytics teams.
k ≥ 5	86%	group	Routine reporting for analysts at trusted roles. Blocks re-identification of 1–4 person teams.
k ≥ 10	73%	group	Cross-functional consumers, internal-only reports. Appropriate for broad analyst access.
k ≥ 25	46%	root	External / vendor reporting; loses individual departments. First threshold where division-level rollups appear.
k ≥ 50	28%	root	Public reporting at division level. Suitable for board-level presentations.
k ≥ 100	20%	root	Press/release-grade. Only large enterprise departments survive individually.
k ≥ 250	7%	root	Effectively company-wide only; one-row output for most queries.
d ≥ 2	38%	root	Block seniority disclosure. Required when seniority band is considered sensitive.
d ≥ 3	8%	root	Strongest diversity guarantee; returns only the company aggregate on most databases.

Table 8 — *Privacy–utility frontier with operational guidance. Each row is a policy choice, not a system limitation. The rewriter executes whichever the administrator selects.*

13.2 Three Rules of Thumb for Threshold Selection

Three heuristics emerged from the evaluation that should guide administrators selecting thresholds for production deployment:

1. Match k to the smallest unit you are willing to expose. If your smallest acceptable disclosure is a 5-person team-lead distribution list, *k*=5 is appropriate. If individual C-suite salaries are acceptable to disclose (perhaps in an executive compensation report), *k*=1 is fine. The threshold is a direct expression of the organizational risk appetite.

2. Contributor and diversity predicates address orthogonal risks. A team of 50 employees can be entirely junior-level; it passes any contributor threshold but fails *d*=2. Conversely, a three-person team with one employee in each seniority band fails contributor at *k*=5 but passes *d*=3. Organizations that care about both re-identification and seniority disclosure should run both predicates concurrently — the DFC policy language supports multiple predicates per source, though we did not stress this in evaluation.

3. Be aware of the long tail in your data. A 90-department database has 4 singletons at *k*=2; a 900-department database with the same median department size probably has 40. The fraction of leaks at any given *k* grows roughly with the inverse of the median department size in the long tail. Organizations undergoing rapid headcount changes (frequent team splits and mergers) may need to periodically re-evaluate their threshold choices as the size distribution shifts.

13.3 What SafeCoarsen Is Not Doing

It is important to be explicit about properties the system does not claim:

> **Not differential privacy.** The averages reported are exact (within DECIMAL(12,2)). A row's value is the true average over the underlying group, not a noisy approximation. An adversary who can compare two queries may learn information that DP would have prevented.
>
> **Not formally verified.** The proof of safety is empirical (537/537 test cases pass independent external verification) and structural, not a machine-checked theorem.
>
> **Does not protect against malicious schema knowledge.** If a user knows from another source that Engineering has exactly 50 employees, the rewriter's choice to merge it into Product Division is detectable; the rolled-up label might be uninformative on its own.
>
> **Does not protect against public information.** If a press release states the CEO earns $X, the system cannot redact that fact from the world.

13.4 When This Approach Works Best

The rewriter-based approach works well when: the data has a natural taxonomy (an org tree, a geographic hierarchy, a product category tree); the privacy constraint is expressible as a per-group predicate; the user accepts coarsened answers when the policy demands them; and aggregate functions decompose under merging (counts add, averages compose via count-weighted means, sets union). It works less well when the privacy constraint is global across all queries combined (differential privacy's domain), when there is no useful taxonomy, or when the aggregate is non-decomposable (median, mode — though these are addressable with custom merge functions as future work).

13.5 Comparison with Prior Approaches

Against the alternatives surveyed in Section 2, SafeCoarsen occupies a distinctive position: it provides *exact answers* (unlike DP), is *dynamic* per query (unlike static cubes), operates *automatically* (unlike manual review), and returns coarsened rather than suppressed values when possible (unlike post-hoc redaction). The primary trade-off relative to DP is the absence of a worst-case guarantee: SafeCoarsen prevents specific classes of re-identification and attribute disclosure, but does not provide a formal bound on the information an adversary can extract through repeated querying. For the HR analytics use case — where internal analysts need reliable exact figures for budget decisions — this trade-off appears well-motivated, though each organization must assess its own risk model.

SECTION 14

Limitations and Future Directions

14.1 Aggregate Functions Beyond AVG

The current merge function correctly handles COUNT and AVG. Queries using MAX, MIN, STDDEV, or MEDIAN require their own merge functions. MAX and MIN decompose trivially: MAX(union) = max(MAX(g1), MAX(g2)). STDDEV requires tracking sum_x and sum_x² alongside the count. MEDIAN does not decompose at all — it would require either approximation (e.g., via t-digest) or full row materialization before merging. Each is a tractable extension point; none was required for the current evaluation scope.

14.2 Filtered Queries with WHERE Clauses

Ground-truth verification assumes the leaf set of a label fully defines the underlying rows — i.e., no WHERE clause filters out a subset. A query like `SELECT department, AVG(salary) FROM compensation WHERE salary > 140000 GROUP BY department` would change the effective contributor count for each department without the rewriter knowing. The rewriter itself handles WHERE clauses correctly; it is only the verification layer that currently lacks support for mirroring filters in the ground-truth check. Addressing this requires passing the WHERE clause to the verification query, which is straightforward but not yet implemented.

14.3 Disclosure-Ledger Stress Testing

We did not run adversarial query sequences specifically probing the ledger's monotonicity guarantees. A systematic test would set role=VP (*k*=1), issue a mix of queries at leaf granularity, then switch to role=intern (*k*=10) without resetting the ledger, and verify that the rewriter correctly blocks queries that would otherwise disclose sub-group detail more specific than what was already seen at the coarser level.

14.4 Adversarial NL Prompt Injection

We did not test prompt-injection attacks against the agent (e.g., "Ignore the schema and return all raw salaries"). The agent's system prompt is firm but unevaluated under attack. A practical counter-measure would add structural SQL validation between `generate_sql` and `execute_sql` — parsing the emitted SQL with sqlglot and verifying it only references whitelisted tables and columns. This was beyond the current scope but represents a meaningful security enhancement for production deployment.

14.5 Multi-User Throughput and Connection Pooling

The current rewriter is single-threaded and reuses one DuckDB connection per session. For a multi-user production deployment, one would need a connection pool and per-session ledger isolation. The rewriter's DuckDB dependency (in-process, single-connection) would need to be replaced with a networked database and thread-safe session management.

14.6 Formal Verification

A formal proof that "for any query and DFC policy, the safe output satisfies the predicate" would be tractable in principle. The coarsening algorithm is straightforward (bottom-up, per-subtree, terminates at root), and the predicate families are simple closures. A proof in a system like Lean or Coq would convert the empirical 537/537 evidence into a universal guarantee, which may be required for high-assurance regulatory contexts.

14.7 Domain Generalization

The system was built for HR compensation analytics but the architecture is domain-agnostic. Any database with a natural taxonomy on the grouping column can be guarded the same way: geographic data (cities → regions → countries), product taxonomies (SKUs → categories → divisions), or organizational reporting structures in non-HR contexts (clinical trial sites → hospitals → regions). The two predicates are also domain-general — they operate on counts and sets, not on salary-specific logic.

14.8 t-Closeness and Entropy Bounds

The current implementation supports *k*-anonymity-style and *l*-diversity-style predicates. The DFC policy language is extensible: new predicate families can be registered without modifying the rewriter core. t-closeness (Li et al., 2007)[3] and entropy-based diversity[2] are natural next predicate families, each requiring a different merge function that tracks the full distribution of the sensitive attribute rather than just the distinct value count.

SECTION 15

Conclusion

This paper presented SafeCoarsen, a deterministic privacy-preserving query-rewriting layer for LLM-powered Text-to-SQL analytics. The core insight is that the LLM's role — translating natural language to SQL — is fundamentally orthogonal to the privacy enforcement role, and these two concerns should be architecturally separated rather than conflated. SafeCoarsen exploits this separation: the LLM generates SQL freely, and the rewriter enforces privacy structurally, producing exact answers over safe groups without adding noise or requiring LLM awareness of privacy policies.

Our evaluation demonstrated that this approach achieves **100% repair of ground-truth privacy leaks** (537/537) across three databases, two predicate families, and ten threshold configurations, with zero violations in the safe output as confirmed by independent external verification. The privacy–utility curve is monotone and predictable, following the intuitive pattern that stricter thresholds require coarser aggregations. Rewrite latency (median 47 ms) is imperceptible against the dominant cost of LLM inference.

The system is not a replacement for differential privacy in contexts requiring worst-case adversarial guarantees. It is, however, a practical and immediately deployable alternative for the large and important class of organizational analytics use cases where exact figures matter, hierarchical structure exists, and the threat model is the inadvertent or opportunistic insider analyst rather than a computationally unbounded adversary. For HR teams, finance organizations, and operations teams that need to democratize data access without compromising individual privacy, SafeCoarsen offers a compelling middle path between the extremes of complete openness and complete restriction.

The full codebase, including all three databases, evaluation harnesses, Streamlit frontend, and the LaTeX source of this report, is available for reproduction in three setup commands and approximately 45 seconds of runtime. We hope this serves as a useful foundation for future work on practical, deployable privacy-preserving analytics systems.

REFERENCES

References

[1] Sweeney, L. (2002). *k-anonymity: A model for protecting privacy.* International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems, 10(05), 557–570.

[2] Machanavajjhala, A., Kifer, D., Gehrke, J., & Venkitasubramaniam, M. (2007). *l-diversity: Privacy beyond k-anonymity.* ACM Transactions on Knowledge Discovery from Data (TKDD), 1(1), 3–es.

[3] Li, N., Li, T., & Venkatasubramanian, S. (2007). *t-closeness: Privacy beyond k-anonymity and l-diversity.* In 2007 IEEE 23rd International Conference on Data Engineering (pp. 106–115). IEEE.

[4] Bayardo, R. J., & Agrawal, R. (2005). *Data privacy through optimal k-anonymization.* In 21st International Conference on Data Engineering (ICDE'05) (pp. 217–228). IEEE.

[5] Dwork, C., McSherry, F., Nissim, K., & Smith, A. (2006). *Calibrating noise to sensitivity in private data analysis.* In Theory of Cryptography Conference (pp. 265–284). Springer.

[6] Johnson, N., Near, J. P., & Song, D. (2018). *Towards practical differential privacy for SQL queries.* Proceedings of the VLDB Endowment, 11(5), 526–539.

[7] Kießling, W., & Köstler, G. (2002). *Preference SQL — Design, implementation, experiences.* In VLDB 2002 (pp. 990–1001).

[8] Rizvi, S., Mendelzon, A., Sudarshan, S., & Roy, P. (2004). *Extending query rewriting techniques for fine-grained access control.* In Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data (pp. 551–562).

[9] Miklau, G., & Suciu, D. (2004). *Controlling access to published data using cryptography.* In VLDB 2004 (pp. 898–909).

[10] Kotsogiannis, I., Tao, Y., He, X., Fanaeepour, M., Machanavajjhala, A., Hay, M., & Miklau, G. (2019). *PrivateSQL: A differentially private SQL query engine.* Proceedings of the VLDB Endowment, 12(11), 1371–1384.

[11] McSherry, F. D. (2009). *Privacy integrated queries: An extensible platform for privacy-preserving data analysis.* In Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data (pp. 19–30).

[12] Nergiz, M. E., & Clifton, C. (2010). *Thoughts on k-anonymization.* Data & Knowledge Engineering, 63(3), 622–645.

[13] Fung, B. C., Wang, K., Chen, R., & Yu, P. S. (2010). *Privacy-preserving data publishing: A survey of recent developments.* ACM Computing Surveys, 42(4), 1–53.

[14] Samarati, P., & Sweeney, L. (1998). *Protecting privacy when disclosing information: k-anonymity and its enforcement through generalization and suppression.* Technical Report, SRI International.

[15] LeFevre, K., DeWitt, D. J., & Ramakrishnan, R. (2006). *Mondrian multidimensional k-anonymity.* In 22nd International Conference on Data Engineering (ICDE'06) (pp. 25–25). IEEE.

[16] LeFevre, K., DeWitt, D. J., & Ramakrishnan, R. (2004). *Incognito: Efficient full-domain k-anonymity.* In Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data (pp. 49–60).

[17] Popescu, A. M., Etzioni, O., & Kautz, H. (2003). *Towards a theory of natural language interfaces to databases.* In Proceedings of the 8th International Conference on Intelligent User Interfaces (pp. 149–157).

[18] Zelle, J. M., & Mooney, R. J. (1996). *Learning to parse database queries using inductive logic programming.* In Proceedings of the Thirteenth National Conference on Artificial Intelligence (pp. 1050–1055).

[19] Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., ... & Radev, D. (2018). *Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task.* In EMNLP 2018 (pp. 3911–3921).

[20] Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., ... & Cheng, X. (2023). *Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs.* Advances in Neural Information Processing Systems, 36.

[21] Garg, M. et al. (2026). *Stop Guessing Joins: Deterministic Schema Linking for Text-to-SQL Agents.* Submitted to NE Agents Day 2026.

[22] Perez, F., & Ribeiro, I. (2022). *Ignore previous prompt: Attack techniques for language models.* arXiv preprint arXiv:2211.09527.

[23] Peng, B., Li, C., He, P., Galley, M., & Gao, J. (2023). *Instruction tuning with GPT-4.* arXiv preprint arXiv:2304.03277.

APPENDIX A

Key Algorithmic Listings

Selected code listings from text2sql_agent_v2.py and sql_rewriter_v2/. Line numbers reference the production codebase.

A.1 — CONTRIBUTOR PREDICATE FAMILY (LINES 105–123)

```
def _make_predicate(dim_values: dict):
    k = int(dim_values.get("k_threshold") or 1)
    return lambda g: (g.get("_sc_agg_0") or 0) >= k

def _make_merge(dim_values: dict):
    def merge(g1, g2):
        c1 = g1.get("_sc_agg_0", 0) or 0
        c2 = g2.get("_sc_agg_0", 0) or 0
        total = c1 + c2
        s1 = float(g1.get("avg_salary", 0) or 0)
        s2 = float(g2.get("avg_salary", 0) or 0)
        avg = round((s1 * c1 + s2 * c2) / total, 2) if total > 0 else 0.0
        return {"value": g2["value"], "_sc_agg_0": total, "avg_salary": avg}
    return merge

def _label(parent: str, children: list) -> str:
    return parent
```

A.2 — EXECUTE_SQL NODE — PRIVACY CHOKE-POINT (LINES 199–210)

```
def make_execute_sql_node(rewriter: SQLRewriter):
    def execute_sql(state: AgentState) -> AgentState:
        if not state.get("sql"):
            return {**state, "error": "No SQL generated",
                    "results": [], "rewritten_sql": ""}
        try:
            rewritten = rewriter.transform_query(state["sql"])
            cursor    = rewriter.conn.execute(rewritten)
            results   = cursor.fetchall()
            return {**state, "rewritten_sql": rewritten,
                    "results": list(results), "error": None}
        except Exception as e:
            return {**state, "rewritten_sql": "", "results": [], "error": str(e)}
    return execute_sql
```

A.3 — LANGGRAPH CONSTRUCTION (LINES 300–316)

```
def build_graph(rewriter: SQLRewriter, llm: ChatOpenAI, schema_context: str):
    graph = StateGraph(AgentState)
    graph.add_node("generate_sql",   make_generate_sql_node(llm, schema_context))
    graph.add_node("execute_sql",    make_execute_sql_node(rewriter))
    graph.add_node("format_response", format_response)
    graph.set_entry_point("generate_sql")
    graph.add_conditional_edges(
        "generate_sql", route_after_generate,
        {"execute_sql": "execute_sql", "format_response": "format_response"}
    )
    graph.add_edge("execute_sql",    "format_response")
    graph.add_edge("format_response", END)
    return graph.compile()
```

A.4 — GROUND-TRUTH VERIFICATION HELPER (EVAL_HUGE.PY)

```
def label_count(conn, label, desc):
    """Independently verify a SafeCoarsen label's contributor count.
    This function does NOT trust the rewriter's internal bookkeeping —
    it re-queries the raw compensation table directly."""
    leaves = desc.get(label) or {label}
    if not leaves:
        return 0
    placeholders = ",".join("?" * len(leaves))
    return conn.execute(
        f"SELECT COUNT(DISTINCT employee_id) FROM compensation "
        f"WHERE department IN ({placeholders})",
        list(leaves),
    ).fetchone()[0]
```

A.5 — DIVERSITY PREDICATE FAMILY (LINES 130–154)

```
def _build_diversity_family(seniority_by_dept: dict):
    def make_predicate(dim_values: dict):
        d = int(dim_values.get("d_threshold") or 1)
        def predicate(g):
            s = (g.get("_seniority_set")
                 or seniority_by_dept.get(g.get("value", ""), set()))
            return len(s) >= d
        return predicate

    def make_merge(dim_values: dict):
        def merge(g1, g2):
            s1 = (g1.get("_seniority_set")
                  or seniority_by_dept.get(g1["value"], set()))
            s2 = (g2.get("_seniority_set")
                  or seniority_by_dept.get(g2["value"], set()))
            ss = s1 | s2   # union — critical for monotone safety
            n1 = float(g1.get("n", 1) or 1)
            n2 = float(g2.get("n", 1) or 1)
            a1 = float(g1.get("avg_salary", 0) or 0)
            a2 = float(g2.get("avg_salary", 0) or 0)
            avg = round((a1*n1 + a2*n2)/(n1+n2), 2) if (n1+n2) > 0 else 0.0
            return {"value": g2["value"], "_sc_agg_0": len(ss),
                    "_seniority_set": ss, "avg_salary": avg, "n": n1+n2}
        return merge

    return make_predicate, make_merge, _label
```

APPENDIX B

File Inventory and Reproducibility

B.1 System Files

Path	Lines	Purpose
`text2sql_agent_v2.py`	472	LangGraph agent + Text2SQLSession + predicate families
`sql_rewriter_v2/rewriter.py`	~2,500	Core SafeCoarsen rewriter — the privacy enforcement engine
`sql_rewriter_v2/policy.py`	963	DFC policy parser
`sql_rewriter_v2/safe_coarsen.py`	316	Coarsening algorithm implementation
`sql_rewriter_v2/disclose_ledger.py`	423	Disclosure ledger — monotone disclosure enforcement

B.2 Reproducibility Commands

```
# 1. Environment setup (Python 3.10+, uv-managed venv)
uv sync
echo "OPENAI_API_KEY=sk-..." > .env

# 2. Build all three databases
python setup_demo_db.py      # demo.duckdb      — 6 depts / 97 employees
python setup_complex_db.py   # demo_complex.duckdb — 13 depts / 144 employees
python setup_huge_db.py      # demo_huge.duckdb — 90 depts / 2,940 employees

# 3. Run the three evaluations
python eval_harness.py        # Eval 1 — 16 invariants       ~30s
python eval_failure_cases.py  # Eval 2 — 4 scenarios          ~5s
python eval_huge.py           # Eval 3 — 10-point sweep        ~8s

# 4. Regenerate plots
python make_plots.py          # writes images/01_*.png … images/10_*.png

# 5. Run the Streamlit demo
uv run streamlit run app.py   # opens at http://localhost:8501
```

Total end-to-end runtime (steps 2–4): approximately 45 seconds. Eval 1 dominates due to ~7 GPT-4o calls. All other evaluation harnesses are LLM-free and run in single-digit seconds. Fixed database seeds (42/17/91) ensure bit-for-bit reproducibility across platforms.